

PROJECT REPORT: AUTOMATED BANK MANAGEMENT & ATM SYSTEM

Platform: VITyarthi - Build Your Own Project

1. Cover Page

Project Title: Automated Bank Management & ATM System

Student Name: Ved Naman

Registration Number: 25BAI10163

Course: JAVA

Date: September 17, 2026

2. Introduction

Modern banking relies on automated systems to ensure speed, accuracy, and accessibility for users. This project is a comprehensive desktop-based Bank Management and ATM Simulation application built using Java Swing and JDBC. It provides an intuitive Graphical User Interface (GUI) to handle multi-step customer registration, secure authentication, and core banking transactions such as cash withdrawals, deposits, and balance checks, with all data persistently stored in a MySQL relational database.

3. Problem Statement

Traditional bank branch operations often suffer from slow processing times, clerical errors, and lack of real-time auditability. Customers requiring basic services like cash deposits, withdrawals, or statements are subjected to long wait times. This project addresses these inefficiencies by offering a robust, 24/7 automated teller simulation that instantly validates operations and maintains a centralized, tamper-resistant transaction ledger.

4. Functional Requirements

- **Customer Registration:** A three-tier form (Signup 1, 2, 3) capturing personal, demographic, and account details.
- **Authentication:** Card number and 4-digit PIN verification before granting access to the dashboard.
- **Transaction Handling:** Modules for Deposits, Withdrawals (with overdraw protection), and preset Fast Cash options.
- **Account Auditing:** Real-time Balance Enquiry and Mini Statement generation tracking past transactions.
- **Account Settings:** Secure PIN modification facility.

5. Non-functional Requirements

- **Usability:** A clean, realistic ATM interface utilizing Java Swing components.
- **Data Integrity:** Strict validation to prevent negative balances and ensure atomic transaction logs.
- **Security:** PIN validation required for critical actions (withdrawal, fast cash, PIN change).
- **Maintainability:** Modular codebase separating database connections (Connn.java) from GUI screens.

6. System Architecture

The application adopts a 3-tier client-server architecture:

1. **Presentation Layer (GUI):** Java Swing and AWT classes (JFrame, JButton) that capture user inputs.
2. **Business Logic Layer:** Event handlers (ActionListener) that process rules, calculate balances, and route screens.
3. **Data Access Layer (Database):** JDBC Driver (MySQL Connector) executing SQL queries to read/write from the 'bankSystem' tables.

7. Design Diagrams

- **Use Case Diagram:** Actors (Customer) can register, login, deposit, withdraw, check balance, and change PIN.
- **Workflow Diagram:** Welcome Screen -> Login -> Main Menu -> Selected Transaction -> Success/Failure Prompt -> Return to Main Menu.
- **Sequence Diagram:** Customer enters amount -> Withdraw.java validates balance -> Connn.java queries DB -> If balance > amount, execute debit -> Return Success.
- **Class Diagram:** Main GUI classes (Login, Signup, Deposit) inherit from JFrame and implement ActionListener. Connn serves as the global JDBC helper.
- **ER Diagram (Storage):** Tables 'signup', 'signuptwo', 'signupthree' linked by form_no. 'login' linked by card_number. 'bank' ledger linked by pin.

8. Design Decisions & Rationale

- **Java Swing over CLI:** Chosen to accurately simulate a visual ATM kiosk, providing a much better user experience.
- **MySQL Relational Database:** Chosen over flat files to maintain referential integrity between customer profiles, login credentials, and transaction ledgers.
- **Transaction Logging:** Instead of overwriting a single balance column, every deposit and withdrawal is logged as a separate row in the 'bank' table. The balance is dynamically aggregated to provide a complete audit trail (Mini Statement).

9. Implementation Details

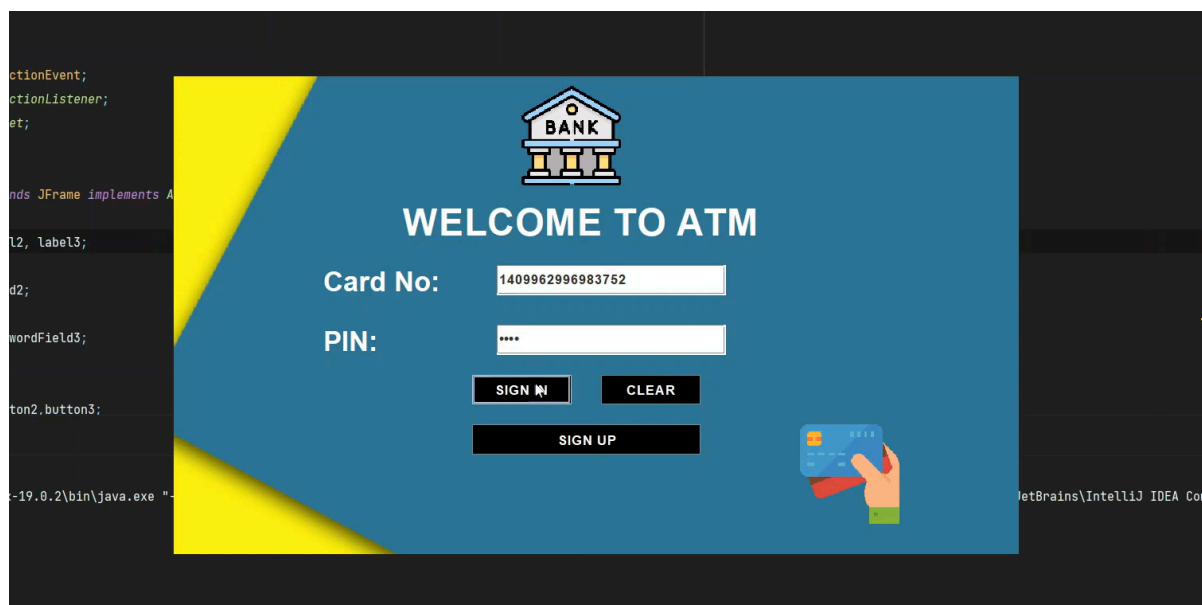
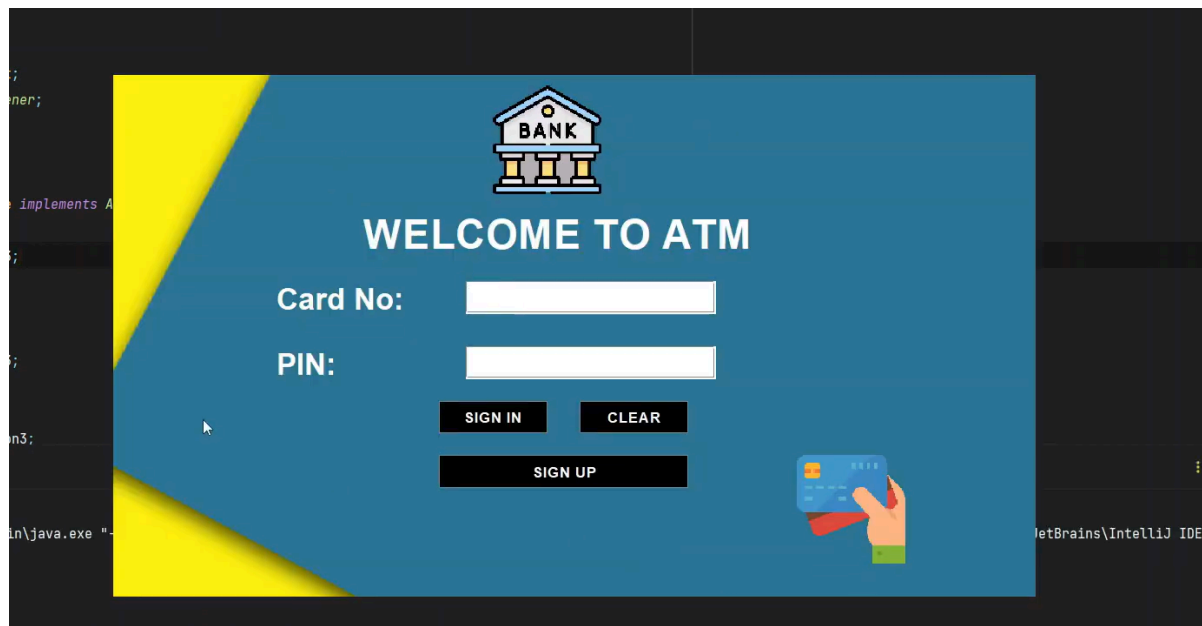
The project contains 12 core Java classes:

- **Connn.java:** Establishes the JDBC connection.
- **Signup.java, Signup2.java, Signup3.java:** Registration screens.
- **Login.java:** Authentication gateway.
- **main_Class.java:** The central ATM dashboard.
- **Deposit.java & Withdrawl.java:** Financial mutators.
- **FastCash.java:** Quick withdrawal module.

- **BalanceEnquiry.java & mini.java:** Read-only ledger queries.
- **Pin.java:** Updates credentials.

10. Screenshots / Results

Below is the ATM Main Dashboard output showing the real-time balance and transaction options





11. Testing Approach

Manual testing was conducted across key operational paths:

- **Positive Testing:** Successful login with correct PIN, successful deposit reflecting in the database.
- **Negative Testing:** Attempting to withdraw funds exceeding the available balance (verifying the "Insufficient Balance" pop-up). Entering alphabets in numeric fields.
- **Edge Cases:** Zero-value deposits and multi-screen navigation memory leaks.

12. Challenges Faced

- **State Management:** Managing multiple Swing JFrames required precise control of setVisible() and dispose() to avoid orphaned background windows.
- **Dynamic Balances:** Fetching and calculating the user's total balance on-the-fly required precise string parsing of historical DB entries.

13. Learnings & Key Takeaways

- Mastered Java Swing layout managers and event-driven programming.
- Deepened understanding of JDBC, database connections, and SQL operations.
- Learned how to structure a medium-scale Java project into logical application flows.

14. Future Enhancements

- Upgrade SQL queries from Statement to PreparedStatement to prevent SQL Injection.
- Implement SHA-256 hashing for storing PINs securely in the database.
- Add receipt generation by exporting statements to PDF.

15. References

- Java Platform, Standard Edition Documentation (Oracle).
- MySQL Connector/J Developer Guide.
- VITyarthi Project Guidelines.